Title

Analysis of Recursive Stack Depth in Merge Sort

Aim

To analyze the effect of recursive stack depth in Merge Sort on systems with limited memory, measure stack usage and execution time, and study how recursion affects resource utilization.

Procedure

1. Read the input array.
2. Apply the Merge Sort algorithm recursively.
3. Record the maximum recursion (stack) depth.
4. Measure the execution time.
5. Display the sorted array, stack depth, and execution time.
6. Analyze the impact of recursion on memory usage.
7. Suggest optimization techniques to reduce stack overhead.

Algorithm

Step 1: Start.

Step 2: Read the input array.

Step 3: Divide the array into two equal halves recursively until each subarray contains one element.

Step 4: Merge the sorted subarrays.

Step 5: Record the maximum recursion depth during execution.

Step 6: Measure the execution time.

Step 7: Display the sorted array, stack depth, and execution time.

Step 8: Stop.

Input

Array: 38 27 43 3 9 82 10

Output

Sorted Array: [3, 9, 10, 27, 38, 43, 82]

Maximum Stack Depth: 4

Execution Time: 0.08 ms

Time Complexity = Θ(n log n)

Stack Space = Θ(log n)

Analysis

- Merge Sort divides the array recursively into smaller subarrays.
- The recursion depth increases approximately as $\log_2(n)$.
- Although Merge Sort has an efficient **Θ(n log n)** time complexity, recursive function calls consume stack memory.
- For very large datasets or systems with limited memory, excessive recursion may increase stack usage.
- Merge Sort is more memory-efficient than many recursive algorithms because its recursion depth grows logarithmically rather than linearly.

Result

The Merge Sort algorithm successfully sorted the input array while recording the recursion stack depth and execution time. The algorithm has a **time complexity of Θ(n log n)** and requires **Θ(log n)** stack space due to recursive calls. Its logarithmic stack growth makes it suitable for most applications, but iterative implementations can further reduce stack overhead in memory-constrained systems.